Singly Linked List – Search Element In Sorted List Recursive and Time Complexity.

Program:

```
bool searchElementInSortedListRecursive(Node **head, int key)
{
    if (*head == nullptr)
    {
        return false;
    }
    else
    {
        if ((*head)->data == key)
        {
            return true;
        }
        else
        {
            return searchElementInSortedListRecursive(&((*head)->next), key);
        }
    }
}
…..
case 11:
        {
            int key;
            cout << "Enter the element to search: ";
            cin >> key;

            if (searchElementInSortedListRecursive(&head, key))
            {
                cout << "Element found." << endl;
            }
```

```
        else
        {
            cout << "Element not found or List is Empty."<< endl;

        }

    }
```

Space Complexity

Input Space:

A. Parameter – Only Definition:

- ***The input space consists of the parameters passed to the function: Node ** head and int key.***
- ***Both of these parameters require constant space, i.e., $O(1)$.***

This view only considers the memory for the parameters passed to the function. The parameters are `Node **head` (a pointer-to-a-pointer) and `int key` (an integer), both of which have a **fixed, constant size**. Therefore, the input space complexity is **O(1)**.

- **Node **head – one pointer (machine word)**
- **int key – one integer (machine word)**

⇒ Θ(1) (= O(1))

B. Full – Data Definition:

This more common definition includes the entire data structure. If the linked list has **n nodes**, the space required to store that list is proportional to `n`. Therefore, the input space complexity is **Θ(n)**.

- ***Same two words as above (Θ(1)), plus***
- ***Every node reachable from *head when the call starts.***

If the list contains n nodes, they occupy Θ(n) bytes.

⇒ Θ(n) (= O(n))

Auxiliary Space:

- ***The auxiliary space is the extra space used by the function during its execution, excluding the input space.***

- ***In this recursive function, each recursive call adds a new frame to the call stack. The maximum depth of recursion is equal to the number of nodes in the linked list in the worst − case scenario.***

- ***Therefore, the auxiliary space complexity due to recursion is $O(N)$, where N is the number of nodes in the linked list.***

Auxiliary space is the extra memory an algorithm uses, not counting the input. For a recursive function, the primary source of auxiliary space is the **call stack**.

1. **Call Stack Usage**: Each time the function calls itself, the system uses a small, constant amount of memory on the call stack. This "stack frame" stores the function's parameters (`head` and `key`) and the return address.
2. **Recursion Depth**: In the worst-case scenario (the `key` is the last element or is not in the list), the function will call itself recursively for every single node. If the list has `n` nodes, this results in a chain of `n` function calls.
3. **Total Space**: The total auxiliary space is the size of one stack frame (which is constant, O(1)) multiplied by the maximum depth of the recursion (which is `n` in the worst case).

This means the space required by the call stack grows linearly with the number of nodes in the list.

Therefore, the auxiliary space complexity is **Θ(n)**.

Auxiliary (extra) Space Complexity

What the algorithm itself adds beyond its input:
• Locals per call: a couple of pointers/booleans → Θ(1) per recursion level.
• Recursion stack: in the worst case (key not found, or found in the last node) the routine makes one call per node, i.e. depth = n.
– Total stack space = depth × Θ(1) = Θ(n).
• No dynamic-heap allocations are performed.
⇒ Worst-case auxiliary space: Θ(n) (= O(n))

(If the key is found after k nodes, only k stack frames are created, so the auxiliary space is Θ(k); but asymptotically, the upper bound is linear in n. A compiler that performs guaranteed tail-call optimisation could reduce this to Θ(1), yet such optimisation is not required by the C++ standard, so Θ(n) is the safe assumption.)

Algorithm

$SrchElemINSortdListRec(START, KEY)$

$1.\ [LIST\ is\ Empty?]$

$If\ START = NULL, then:$

$Write:$ ``Error: List is empty.``

$Return\ False\ and\ EXIT.$

$[When\ List\ is\ not\ empty, then\ search\ KEY]$

$Else$

$if\ INFO[START] = Key, then:$

$Return\ True\ and\ exit$

$Else$

$Return\ SrchElemINSortdListRec(ADDRESS\ of\ LINK[START], KEY)$

$[Translation\ of\ ``SrchElemINSortdListRec(ADDRESS\ of\ LINK[START], KEY)``]$

$a.\ [Push\ current\ values\ and\ new\ return\ address\ onto\ stacks.]$

$(i)\ Set\ TOP := TOP + 1$

$(ii)\ Set\ STS[TOP] := ADDRESS\ of\ LINK[START]\ [Here\ STS\ refers\ to\ STACK\ for\ START]$

$(iii)\ Set\ STK[TOP] := KEY\ [Here\ STK\ refers\ to\ STACK\ for\ Key]$

$b.\ [Reset\ parameters]$

$Get\ START := ADDRESS\ of\ LINK[START], KEY := KEY$

$c.\ Go\ to\ Step\ 1.$

$[When\ POP\ in\ STACK\ sets\ TOP\ to\ NULL]$

$(a) if\ TOP := NULL, then:$

$(b)\ [Restore\ top\ values\ on\ stack]$

$(i)\ Set\ START := STS[TOP], KEY := STK[TOP]$

$(ii)\ Set\ TOP := TOP - 1$

$(c)\ Exit$

$[END\ OF\ IF\ STRUCTURE]$

$[END\ OF\ IF\ STRUCTURE]$